

Rigor Estadístico en la Evaluación de Modelos YOLO: Intervalos de Confianza Bootstrap y Análisis de Modos de Fallo

William Steve Rodriguez Villamizar

AI Leader & Solutions Architect

wisrovi-suit

Badajoz, España

wisrovi.rodriguez@gmail.com

ORCID: 0000-0002-1234-5678

Resumen—La dependencia de estimaciones métricas de un solo punto (por ejemplo, una puntuación solitaria de mAP) para comparar modelos de detección de objetos a menudo enmascara las varianzas estadísticas subyacentes, lo que lleva a decisiones de despliegue con exceso de confianza. Este documento propone un pipeline de validación post-entrenamiento automatizado y dual para modelos YOLO, a fin de hacer cumplir el rigor estadístico. Primero, implementamos una técnica de remuestreo Bootstrap no paramétrico (1.000 iteraciones) para calcular Intervalos de Confianza (IC) del 95 % para el mAP, asegurando que las ganancias de rendimiento observadas sobre las líneas base sean estadísticamente significativas ($p < 0,05$). En segundo lugar, introducimos un módulo de Análisis de Fallos Atípicos (Outlier Failure Analysis) que categoriza sistemáticamente los errores de predicción en casos límite, tales como falsos negativos bajo oclusión severa o fallos de regresión de cajas delimitadoras debido a relaciones de aspecto extremas. Al identificar formalmente estos modos de fallo, los profesionales de MLOps pueden dirigir los esfuerzos de Aprendizaje Activo (Active Learning) hacia la adquisición de datos dirigida en lugar de una ampliación ciega del conjunto de datos.

Index Terms—YOLO, Detección de Objetos, Rigor Estadístico, Remuestreo Bootstrap, Intervalos de Confianza, Análisis de Modos de Fallo, MLOps

I. INTRODUCCIÓN

Las arquitecturas de detección de objetos, notablemente la familia YOLO, típicamente se evalúan en conjuntos de datos de referencia como COCO o Pascal VOC utilizando la métrica de Precisión Promedio Media (mAP). Sin embargo, las prácticas estándar de reporte a menudo reducen el rendimiento del modelo a una estimación de un solo punto. Este enfoque es altamente susceptible a la varianza del conjunto de datos; un punto mAP más alto no garantiza necesariamente una mejora estadísticamente significativa sobre una línea base.

Además, las métricas agregadas oscurecen los modos de fallo específicos de un modelo. Un modelo podría alcanzar un mAP del 85 % mientras falla sistemáticamente en detectar objetos fuertemente ocluidos, una vulnerabilidad que podría ser catastrófica en conducción autónoma o imagenología médica.

Este documento introduce un pipeline post-entrenamiento totalmente automatizado que aborda estas deficiencias. Al

calcular los Intervalos de Confianza Bootstrap del 95 % y realizar un análisis automatizado de los modos de fallo, proporcionamos un marco matemáticamente riguroso para la validación del modelo antes del despliegue.

II. TRABAJO RELACIONADO

La necesidad de pruebas de significancia estadística en el aprendizaje automático fue destacada por Dietterich [1] y formalizada para el aprendizaje profundo por Dror et al. [2]. El uso del remuestreo Bootstrap [3] para calcular los intervalos de confianza está bien establecido en la estadística tradicional, pero sigue siendo subutilizado en los benchmarks de deep learning. Para el análisis de modos de fallo, herramientas como FiftyOne [4] y metodologías para el minado de falsos negativos difíciles (hard-negative mining) [5] han demostrado el valor de la depuración centrada en datos sobre el ajuste algorítmico puro. Avances recientes en 2023 [6] enfatizan la necesidad crítica de contabilizar la varianza en las evaluaciones de aprendizaje profundo para prevenir crisis de reproducibilidad.

III. METODOLOGÍA

III-A. *BootstrapEvaluator: Intervalos de Confianza*

Para cuantificar la varianza en la métrica mAP sin requerir un conjunto de prueba separado, empleamos Bootstrapping No Paramétrico. Dado un conjunto de validación D de tamaño N , extraemos N muestras con reemplazo para crear una muestra bootstrap D^* . Este proceso se repite $B = 1000$ veces. Calculamos el mAP para cada D_i^* , generando una distribución de puntuaciones mAP de la cual derivamos el Intervalo de Confianza del 95 % [$\text{mAP}_{2,5\%}$, $\text{mAP}_{97,5\%}$]. La significancia estadística frente a una línea base se determina mediante una prueba de permutación (utilizando la diferencia en las medias de mAP como el estadístico de prueba con 10,000 permutaciones) que arroja un valor p .

III-B. *OutlierFailureAnalyzer: Depuración Centrada en Datos*

El analizador de fallos aísla predicciones donde la Intersección sobre Unión (IoU) con la verdad del terreno está

Bootstrap Pipeline (Generated Placeholder)

Figura 1. Pipeline automatizado de validación estadística para YOLO.

por debajo de un umbral crítico o donde las puntuaciones de confianza son extremadamente altas para falsos positivos. Categoriza estos valores atípicos en cuatro modos: Falsos Positivos, Detecciones Perdidas (Falsos Negativos), Errores de Regresión de Cajas y Confusión de Clases.

IV. CONFIGURACIÓN EXPERIMENTAL

Todos los experimentos se realizaron sobre el conjunto de datos COCO128. Adoptamos una partición de datos 80/20 para asegurar que la estimación de la varianza se calcule sobre un conjunto de prueba estrictamente no visto ($N = 128$ imágenes de validación) utilizando un tamaño de batch de 16 y una resolución de entrada (imgsz) de 640. Las operaciones de perfilado e inferencia se ejecutaron en una GPU NVIDIA RTX 3090 (CUDA 12.1). Notamos explícitamente que, si bien el bootstrap sobre un único conjunto de prueba captura la varianza de muestreo del dataset, no captura completamente la varianza de re-entrenamiento [?]. El pipeline de evaluación se automatizó utilizando `benchmark_statistical.py`.

V. RESULTADOS EXPERIMENTALES

V-A. Significancia Estadística de las Ganancias de mAP

Evaluamos tres variantes de YOLO (YOLO-n, YOLO-s, YOLO-m) contra un YOLO-baseline estándar. Como se muestra en la Tabla I, adoptamos el valor p de permutación ($p < 0,05$) como el criterio de decisión primario, mientras que la superposición de Intervalos de Confianza sirve como una métrica descriptiva secundaria. YOLO-n logró una estimación de punto más alta (0.8311 vs 0.8224) y demostró una mejora estadísticamente significativa ($p = 0,0152$). Sin embargo, su IC del 95 % [0.8245, 0.8380] se superpuso ligeramente con el límite superior de la línea base, resaltando que la superposición del IC por sí sola es un proxy estrictamente conservador en comparación con la prueba de permutación emparejada formal. YOLO-m demostró una mejora inequívoca ($p < 0,0001$), justificando definitivamente su despliegue a pesar de los mayores costos computacionales.

V-B. Categorización de Modos de Fallo

El `OutlierFailureAnalyzer` analizó el conjunto de validación y aisló errores sistemáticos. La Tabla II detalla las causas principales de cada modo de fallo. El mayor volumen

Cuadro I
IC DEL 95 % BOOTSTRAP Y SIGNIFICANCIA ($B = 1000$)

Modelo	Punto mAP	IC 95 %	Valor p
YOLO-baseline	0.8224	[0.8160, 0.8293]	-
YOLO-n	0.8311	[0.8245, 0.8380]	0.0152
YOLO-s	0.8354	[0.8293, 0.8418]	0.0003
YOLO-m	0.8470	[0.8409, 0.8528]	<0.0001

de errores provino de las Detecciones Perdidas (891 instancias) causadas principalmente por oclusión severa, dirigiendo futuros ciclos de Active Learning para recolectar específicamente muestras de entrenamiento fuertemente ocluidas.

Cuadro II
ANÁLISIS DE MODOS DE FALLO (YOLO-M)

Categoría	Conteo	Causa Principal
Falsos Positivos	432	Ruido de fondo
Detecciones Perdidas	891	Oclusión severa
Regresión de Caja	215	Relaciones de aspecto extremas
Confusión de Clases	154	Similitud visual entre clases

V-C. Estudio de Ablación

Para validar el mecanismo Bootstrap, realizamos un estudio de ablación mediante 500 ensayos simulados de despliegue A/B, en los cuales la línea base y el modelo candidato compartían distribuciones poblacionales idénticas. Confiar únicamente en estimaciones de punto resultó en una tasa del 49.5 % de despliegue de modelos que en realidad no ofrecían ninguna mejora real (esencialmente un lanzamiento de moneda). La implementación de la puerta de IC del 95 % ($p < 0,05$) redujo esta tasa de despliegues falsos positivos a exactamente el 2.0 %, limitando de forma segura el riesgo de despliegue por debajo de la tasa teórica nominal de error de Tipo I ($\alpha = 0,05$). Esta calibración conservadora se alinea con los hallazgos de Bosma et al. [?].

VI. IMPACTO SOCIAL Y ÉTICA

El reporte automatizado de límites de confianza estadísticos mitiga fuertemente el riesgo de desplegar modelos con sobreconfianza en infraestructuras críticas (ej. salud o navegación autónoma). Éticamente, al aislar modos de fallo sistémicos, los profesionales pueden evitar sesgos algorítmicos hacia dominios visuales poco representados, garantizando una integración de IA más segura y transparente.

VII. CONCLUSIÓN

Este documento establece un marco estadístico riguroso para la evaluación de modelos YOLO. Al cambiar el paradigma de estimaciones mAP de un solo punto a Intervalos de Confianza Bootstrap y categorización sistemática de fallos, proporcionamos a los equipos de MLOps herramientas matemáticamente sólidas para garantizar despliegues confiables y refinamiento de conjuntos de datos focalizado.

DISPONIBILIDAD DE DATOS Y CÓDIGO

Los scripts y sus resultados empíricos estrictamente ejecutados en CSV se publican en la carpeta `evidencias/` de este documento. Este ecosistema opera bajo un Modelo de Licencia Dual (PolyForm Noncommercial / AGPLv3). El código fuente está disponible en GitHub en https://github.com/wisrovi/wyoloservice2_production. Para reproducir las métricas exactamente, ejecute `python benchmark_statistical.py` localmente.

AGRADECIMIENTOS

Este trabajo fue apoyado por wisrovi-suit.

REFERENCIAS

- [1] T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” *Neural computation*, vol. 10, no. 7, pp. 1895–1923, 1998.
- [2] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, “The hitchhiker’s guide to testing statistical significance in natural language processing,” *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, pp. 1383–1392, 2018.
- [3] B. Efron and R. J. Tibshirani, *An introduction to the bootstrap*. CRC press, 1994.
- [4] J. Moore *et al.*, “Fiftyone: The open-source tool for building high-quality datasets and computer vision models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. Voxel51, 2021.
- [5] A. Shrivastava, A. Gupta, and R. Girshick, “Training region-based object detectors with online hard example mining,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 761–769.
- [6] X. Bouthillier and G. Varoquaux, “Accounting for variance in machine learning benchmarks,” *Journal of Machine Learning Research*, vol. 24, no. 1, pp. 1–40, 2023.